

基礎架構即代碼 (IaC)

底層框架移轉工程白皮書

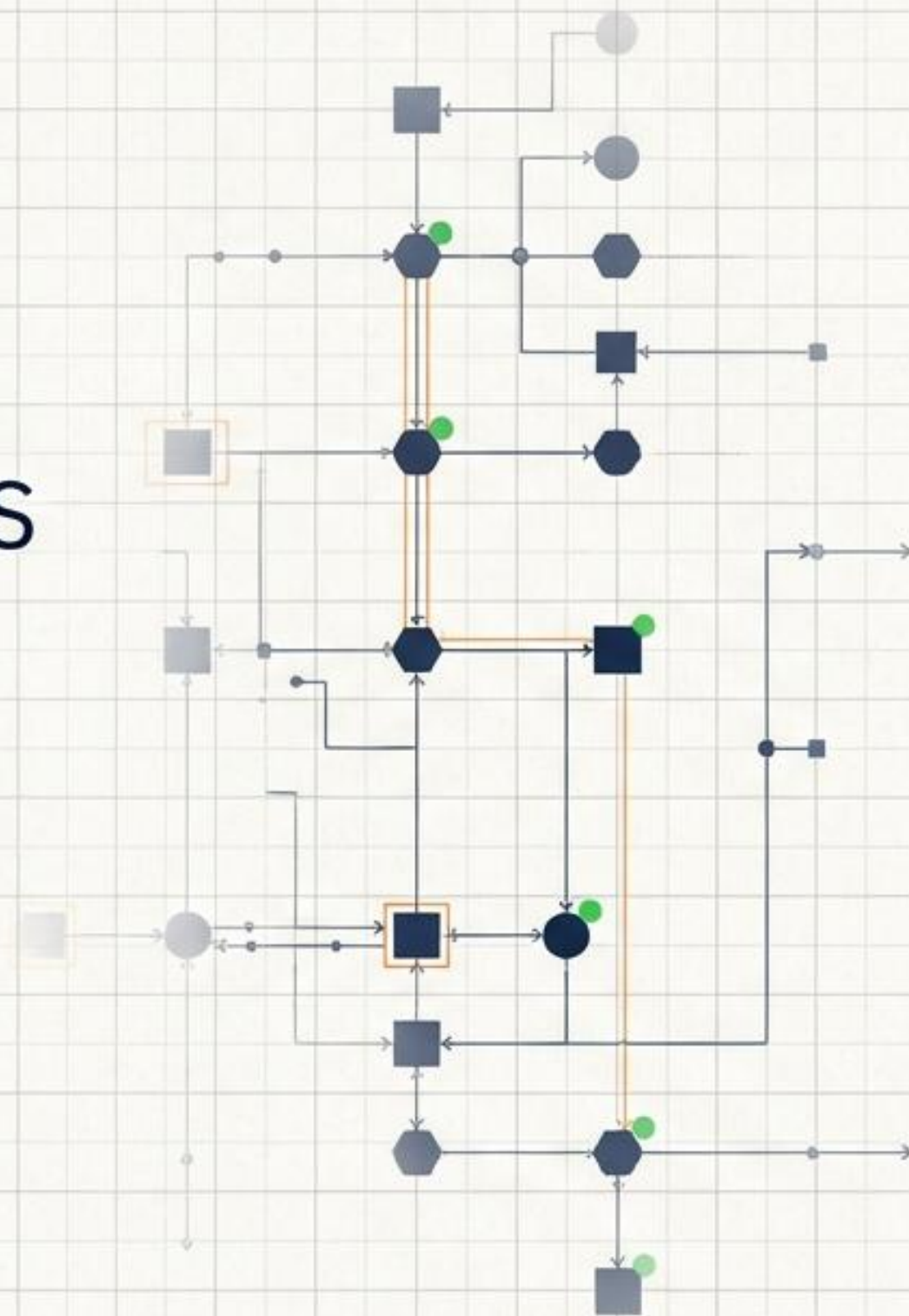
IOT 產品線從 Serverless Framework 至 AWS
CDK 之無痛轉移機制與長期工程價值論證

[DOCUMENT_ID]: REQ-2024-ARCH-001

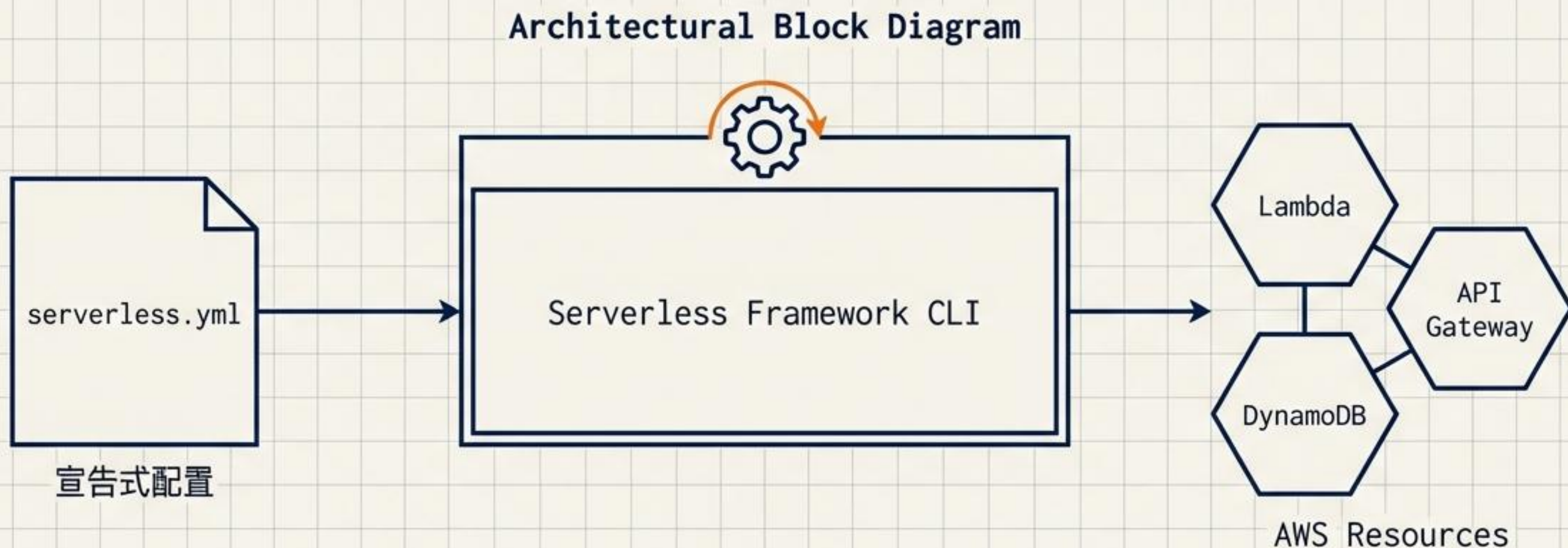
[TARGET_SYSTEM]: IOT Core Infrastructure

[ACTION_TYPE]: Zero-Downtime Migration

[STATUS]: Proposal Ready for Review



緒論：當前 IOT 系統基礎架構觀測模型



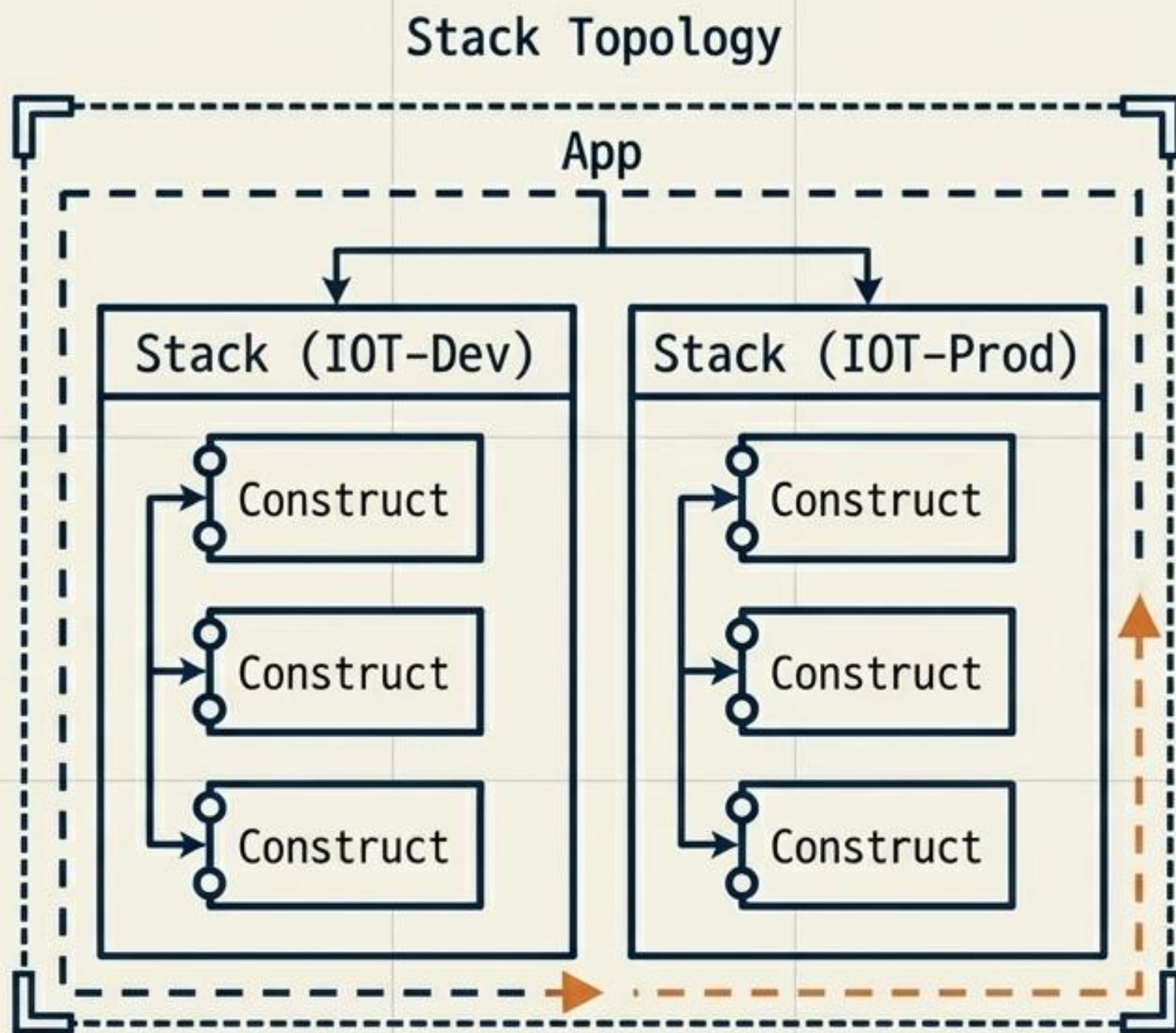
定義	機制	現況
Serverless Framework 為目前 IOT 產品線部署之核心 IaC (Infrastructure as Code) 工具。	透過宣告式的 YAML 設定檔，將抽象架構編譯為具體的雲端資源佈建指令。	系統穩定運行，資源調度精準。然而，外部環境變數的急遽改變，已觸發底層框架的衰退期（詳見次頁風險矩陣）。

風險觀測：現行框架面臨之不可逆衰退指標

2024-12-31

資安與維運風險 (Operational Risk)	商業與財務風險 (Financial Risk)	技術債與相容性風險 (Technical Risk)
		
- 舊版 V.3 已於 2024 年 12 月 31 日全面停止維護 (EOL)。 - 將不再接收任何安全性修補程式 (Security Patches)，直接影響 IOT 系統合規性。	- 新版 V.4 變更授權條款，針對年營收超過 200 萬美元之組織強制收取訂閱費用。 - 若不移轉，基礎設施維護成本將與營收強制綁定，產生非預期的持續性資本支出 (OpEx)。	- AWS 執行環境將淘汰 Python 3.8，而受限於框架限制，升級面臨阻礙。 - 阻斷 IOT 產品線程式碼的自然演進，形成底層架構技術債。

替代方案文獻回顧：AWS 原生開源框架 CDK



[ORIGIN]: AWS 原生支援

由 AWS 官方維護，無須依賴第三方開源社群的維護時程，保障長期穩定性。

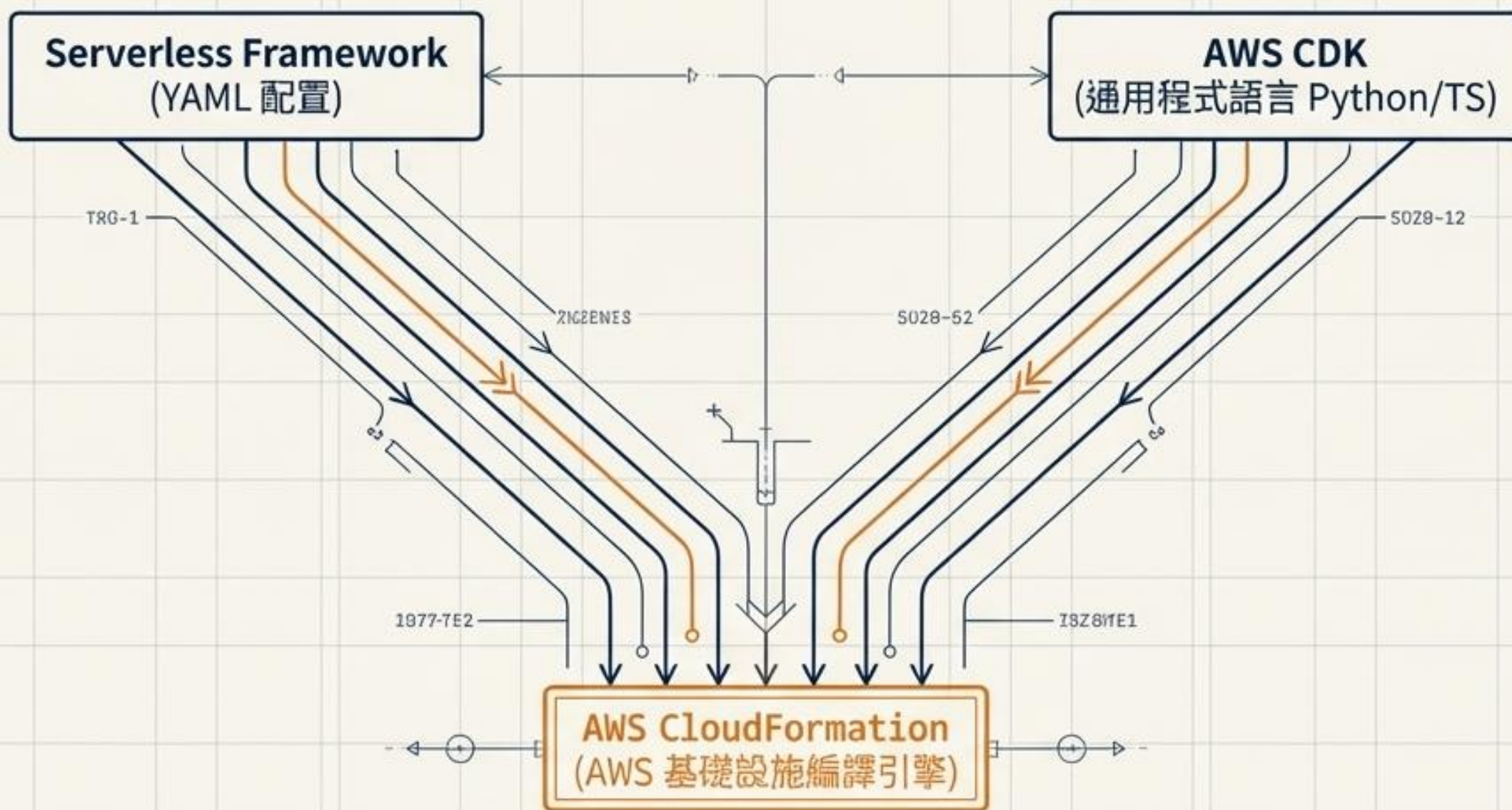
[LICENSE]: 完全開源 (Open-Source)

無任何營收綁定之商業授權條款，徹底消除 \$2M 營收門檻帶來的財務風險。

[LANGUAGE]: 支援通用程式語言

支援 TypeScript, Python, Java 等。允許工程團隊運用迴圈、條件式等標準邏輯架構基礎設施，取代僵化的 YAML 靜態配置。

同源對照實驗：SF 與 CDK 之底層化學反應同構證明



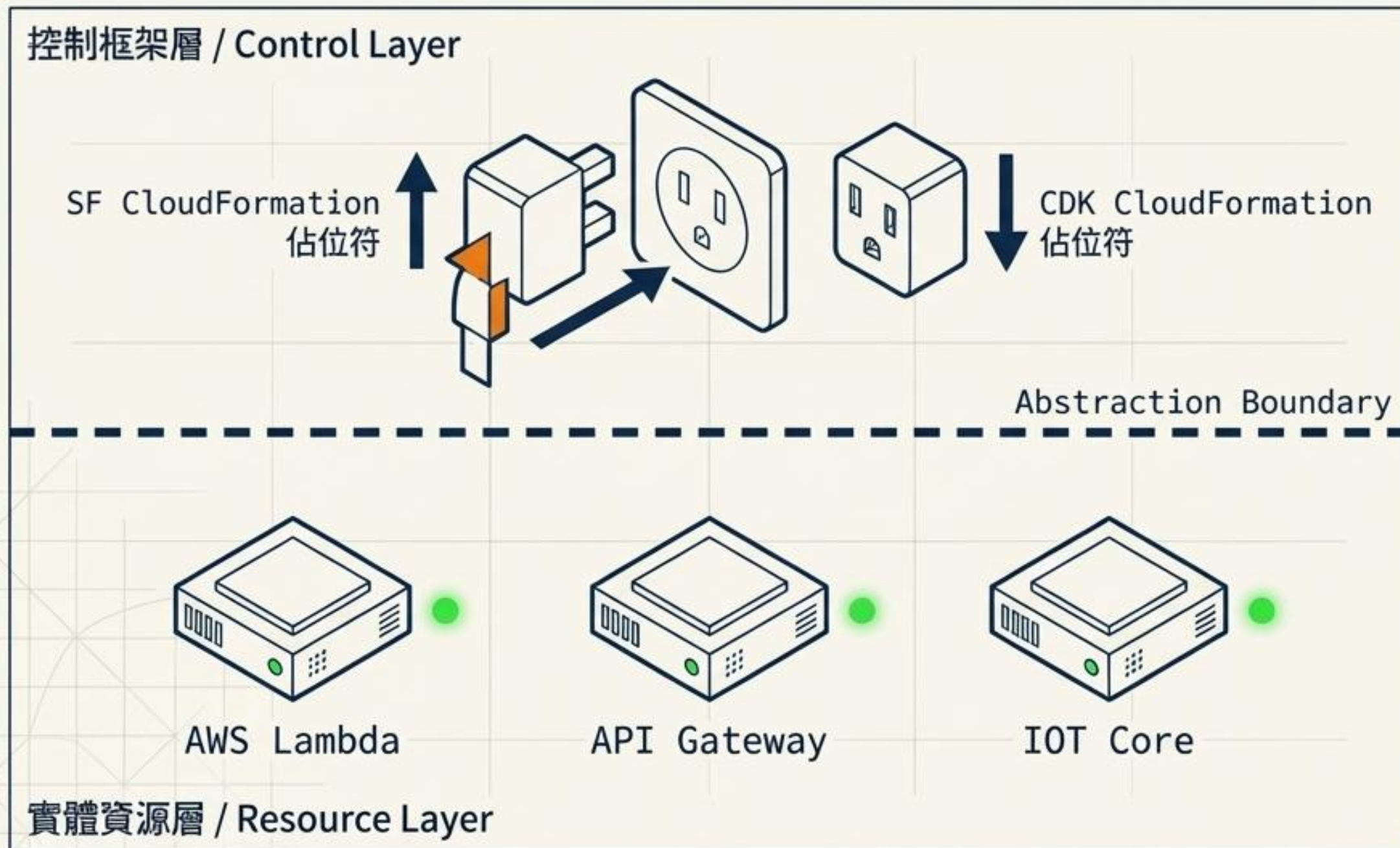
Analytical Takeaway

結論：兩者的核心機制完全相同，最終皆會編譯並轉換為 AWS CloudFormation 範本。

工程意義：這意味著我們並非在「更換雲端資源」，而是單純更換「產生配置檔的遙控器」。此一物理特性為後續的「無痛安全切換」提供了無可辯駁的理論基礎。

外科手術式移轉 (I)：控制層與實體資源之精準剝離

Layered Decoupling Diagram



- 移轉策略核心：嚴格執行「替換遙控器，不碰電視機」的剝離策略。
- 零刪除原則：絕對不刪除任何既有的 AWS 實體運算與網路資源。
- 操作機制：僅針對控制層，拔除 Serverless Framework 所產生的 CloudFormation 佔位符，使實體資源進入準備接受接管的狀態。

外科手術式移轉 (II)：狀態轉移與無縫接管機制

Step 1: 資源保留與舊版卸載 (Retain & Remove)

Action

將原 `CloudFormation Stack` 標記為 `Retain` (保留資源) 後執行刪除。

Status

`IoT` 系統持續服務，毫秒級別零中斷。

Step 2: 新版藍圖重構 (Blueprint Generation)

Action

執行 `CDK` 腳本，產生包含相同實體資源標識符 (Physical IDs) 的新版 `CloudFormation`。

Status

架構定義完成，等待綁定。

Step 3: 資源導入與接管 (CDK Import)

Action

執行 `CloudFormation Import` 操作，將現有實體資源精準對應並寫入新版 `CDK Stack` 中。

Status

接管完成。成功達成無痛切換 (`Seamless Transition`)。

維運平滑性：CloudFormation 熱切換與參數升級

Change Set Comparison Panel

現有狀態 (Current State)

Runtime: Python 3.8

Memory: 512MB

Code: Old App Code

變更集分析 (Change Set)

CDK 透過生成變更集，提前預覽即將修改的參數，確保不涉及資源重建 (Replacement)。

升級狀態 (Upgraded State)

升級執行環境
(Python 3.x 升級完成) ✓

調整記憶體 (Memory
Allocation Upgraded) ✓

抽換程式碼 (New IOT
Code Injected) ✓

所有的設定變更與底層升級，皆遵循 CloudFormation 的動態更新機制，不須暫停服務即可無痛升級，徹底根除 Python 3.8 淘汰之風險。

架構可擴展性：模組化驅動之碎形展開部署

Fractal Expansion Model

基礎環境母版 (Base Construct)

將 IOT 產品線的標準架構封裝為單一可重複使用的程式碼模組。

`cdk deploy`
(單一指令)

[DEV]

[STAGING]

[PROD]

Engineering Value

DRY 原則 (Don't Repeat Yourself)：消除 YAML 時代繁瑣的複製貼上作業。

一致性保障：僅需用同樣的 CDK 腳本執行一次，即可像碎形般快速、無誤差地複製出 N 個完全獨立且規格一致的新產品線環境，大幅提升建置便利性。

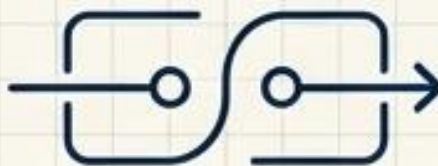
結論：架構演進之綜合工程效益

System Health & Value Dashboard



財務與合規收斂 (Risk & Cost Mitigation)

- 徹底規避 V.4 高達 **\$2M** 營收門檻的授權費用。 ✓
- 擺脫 V.3 EOL 帶來的資安漏洞與合規風險。 ✓



維運連續性 (Operational Continuity)

透過控制層剝離與 CloudFormation Import 機制，保證 IOT 實體連線與服務的 100% 零停機移轉。



開發效能最大化 (Developer Velocity)

以通用程式語言取代靜態配置，透過模組化 (Constructs) 實現基礎環境的一鍵建置與快速迭代。



本次移轉不僅是應對授權變更的防禦性措施，更是確立 IOT 產品線未來三年架構擴展能力之關鍵工程奠基。